LINKORA

Backend V1 Architecture & Implementation Plan

MVC structure • Models • Controllers • Routes • Services • Middleware • Database • API contracts • User flows

DOCUMENT	VERSION	BUILD APPROACH
Backend-first product blueprint	V1.0	Node.js + Express + TypeScript + MongoDB

Product: Linkora, a link-in-bio SaaS platform. A user creates a public profile page and shares one URL containing their profile and important links.

Build principle: Implement the backend in a clear MVC-style structure. Keep business logic in services, database definitions in models, and HTTP handling in controllers and routes.

1. Product definition and V1 scope

Linkora lets a registered user create a public page such as **linkora.app/gobinda**. The user manages profile details, appearance and links through a private dashboard. Visitors can view a published page without creating an account.

V1 features

• Register, login, logout, refresh session and retrieve the current user.

• Create and update one profile per user, choose a unique username, publish or unpublish.

• Create, list, update, reorder, activate/deactivate and delete links.

• Serve a public profile containing only safe profile fields and active links.

• Record basic page-view and link-click events; show an owner-only analytics summary.

• Production basics: validation, centralized errors, authorization, rate limits, logs, API documentation and tests.

Out of scope for V1

• Payments, subscription billing, custom domains, teams and collaboration.

• AI page builder, marketplace, social feed and native mobile apps.

• Arbitrary custom HTML/CSS. V1 uses safe theme presets and allowlisted icons.

2. Backend architecture

```
Client (React frontend later)
        |
        | HTTPS + JSON /api/v1
        v
Express application
  Routes       -> map HTTP method + URL to controller
  Middleware   -> auth, validation, rate limits, request ID, errors
  Controllers  -> read request, call service, send HTTP response
  Services     -> business rules and authorization decisions
  Models       -> Mongoose schemas, indexes and database operations
        |
        v
MongoDB Atlas

Optional integrations:
  Object storage -> profile images
  Email provider -> verification and password reset
  Sentry         -> error monitoring
```

Use a modular MVC organization by feature folder. This is not a microservices architecture. V1 is one deployable Express API, with clear boundaries so it can grow without premature infrastructure.

Recommended backend folder and file structure

```
backend/
■■■ src/
■   ■■■ app.ts
■   ■■■ server.ts
■   ■
■   ■■■ config/
■   ■   ■■■ env.config.ts
■   ■   ■■■ database.config.ts
■   ■   ■■■ cors.config.ts
■   ■   ■■■ cookie.config.ts
■   ■
■   ■■■ models/
■   ■   ■■■ user.model.ts
■   ■   ■■■ profile.model.ts
■   ■   ■■■ link.model.ts
■   ■   ■■■ analytics-event.model.ts
■   ■
■   ■■■ controllers/
■   ■   ■■■ auth.controller.ts
■   ■   ■■■ profile.controller.ts
■   ■   ■■■ link.controller.ts
■   ■   ■■■ public.controller.ts
```

```
■   ■   ■■■ analytics.controller.ts
■   ■
■   ■■■ routes/
■   ■   ■■■ index.routes.ts
■   ■   ■■■ auth.routes.ts
■   ■   ■■■ profile.routes.ts
■   ■   ■■■ link.routes.ts
■   ■   ■■■ public.routes.ts
■   ■   ■■■ analytics.routes.ts
■   ■
■   ■■■ services/
■   ■   ■■■ auth.service.ts
■   ■   ■■■ profile.service.ts
■   ■   ■■■ link.service.ts
■   ■   ■■■ public.service.ts
■   ■   ■■■ analytics.service.ts
■   ■
■   ■■■ middlewares/
■   ■   ■■■ authenticate.middleware.ts
■   ■   ■■■ validate.middleware.ts
■   ■   ■■■ error.middleware.ts
■   ■   ■■■ not-found.middleware.ts
■   ■   ■■■ rate-limit.middleware.ts
■   ■   ■■■ request-id.middleware.ts
■   ■
■   ■■■ validations/
■   ■   ■■■ auth.validation.ts
■   ■   ■■■ profile.validation.ts
■   ■   ■■■ link.validation.ts
■   ■   ■■■ analytics.validation.ts
■   ■
■   ■■■ utils/
■   ■   ■■■ async-handler.ts
■   ■   ■■■ api-response.ts
■   ■   ■■■ app-error.ts
■   ■   ■■■ token.util.ts
■   ■   ■■■ password.util.ts
■   ■
■   ■■■ constants/
■   ■   ■■■ http-status.ts
■   ■   ■■■ app.constants.ts
■   ■
■   ■■■ types/
■       ■■■ express.d.ts
■
■■■ tests/
■   ■■■ unit/
■   ■■■ integration/
■■■ docs/
■   ■■■ openapi.yaml
■■■ .env.example
■■■ package.json
■■■ tsconfig.json
```

Responsibility of each layer

Layer	Responsibility	Must not do
Model	Schema, field types, validation constraints, indexes and persistence.	Send HTTP responses or decide UI behavior.
Route	Declare endpoint, attach middleware and controller.	Contain business logic or database queries.
Controller	Read params/body/user, call service, choose status and response.	Contain complex business rules.
Service	Implement business rules, ownership checks and coordinate models.	Depend on Express request/response objects.
Middleware	Authentication, validation, rate limits, request IDs and error handling.	Silently bypass validation or authorization.
Validation	Define accepted request shape with Zod.	Replace service-level ownership checks.

3. Database design

Use four main MongoDB collections for V1. Store links separately from profiles so link lists can grow and be reordered without rewriting a large profile document.

```
User 1 -------- 1 Profile
  |
  +------------ many Links

Profile 1 ----- many AnalyticsEvents
Link    1 ----- many AnalyticsEvents (for link_click events)
```

3.1 User model: users

Field	Type	Rules
_id	ObjectId	MongoDB primary key
name	String	Required, trimmed, length limited
email	String	Required, lowercase, unique index
passwordHash	String	Required; never expose in API
role	String	user | admin; default user
refreshTokenHash	String | null	Hash only; raw refresh token never stored
createdAt, updatedAt	Date	Mongoose timestamps

3.2 Profile model: profiles

Field	Type	Rules
userId	ObjectId	Required, unique; reference User
username	String	Required, lowercase, unique, URL-safe
displayName	String	Required
bio	String	Optional, max 160 characters
avatarUrl	String	Optional image URL/object key
theme	String	Allowlisted theme ID; default theme
isPublished	Boolean	Default false
createdAt, updatedAt	Date	Mongoose timestamps

3.3 Link model: links

Field	Type	Rules
userId	ObjectId	Required owner reference
profileId	ObjectId	Required profile reference
title	String	Required, max 60 characters
url	String	Required; valid http/https only
icon	String | null	Optional allowlisted icon key
position	Number	Non-negative ordering value
isActive	Boolean	Default true
createdAt, updatedAt	Date	Mongoose timestamps

3.4 AnalyticsEvent model: analytics_events

Field	Type	Rules
profileId	ObjectId	Required profile reference
linkId	ObjectId | null	Null for page_view; link ID for click
eventType	String	page_view | link_click
createdAt	Date	Event timestamp
referrerHost	String | null	Optional hostname only
sessionKeyHash	String | null	Optional pseudonymous dedupe key

Indexes

• users: unique { email: 1 }.

• profiles: unique { username: 1 } and unique { userId: 1 }.

• links: { profileId: 1, position: 1 } and { userId: 1, profileId: 1 }.

• analytics_events: { profileId: 1, createdAt: -1 }.

• Database unique indexes are the final protection against duplicate email/username races.

4. API conventions

Base URL: **/api/v1**. JSON request and response bodies. Use ISO 8601 UTC timestamps. Return string IDs. All protected endpoints require an access token in the Authorization header.

```
Success response:
{
  "success": true,
  "data": { },
  "meta": { }
}

Error response:
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": []
  },
  "requestId": "..."
}
```

4.1 Authentication routes

Method + endpoint	Access	Purpose / body
POST /auth/register	Public	Register: { name, email, password }. Create user + draft profile.
POST /auth/login	Public	Login: { email, password }. Issue access token + refresh cookie.
POST /auth/refresh	Refresh cookie	Rotate refresh token and return a new access token.
POST /auth/logout	Authenticated	Revoke refresh session and clear cookie.
GET /auth/me	Authenticated	Return current safe user and profile summary.
POST /auth/forgot-password	Public	Request reset email; generic response to avoid account enumeration.
POST /auth/reset-password	Reset token	Reset password and revoke existing sessions.

4.2 Profile routes

Method + endpoint	Access	Purpose / body
GET /profiles/me	User	Get own profile.
PATCH /profiles/me	User	Update displayName, bio, avatarUrl, theme.
PATCH /profiles/me/username	User	Change username.
POST /profiles/me/publish	User	Validate required fields, then publish.
POST /profiles/me/unpublish	User	Unpublish own public page.
GET /profiles/username/check?value=...	User	Advisory username availability check.
GET /public/profiles/:username	Public	Get published profile and active links only.

4.3 Link routes

Method + endpoint	Access	Purpose / body
GET /links	User	List own links sorted by position.
POST /links	User	Create: { title, url, icon? }.
PATCH /links/:linkId	User	Update title, url, icon or isActive.
PATCH /links/reorder	User	Reorder: { orderedIds: [id, id, ...] }.
DELETE /links/:linkId	User	Delete owned link.
POST /public/profiles/:username/links/:linkId/click	Public	Record valid click event; 202 accepted.

4.4 Analytics and system routes

Method + endpoint	Access	Purpose
POST /public/profiles/:username/view	Public	Record page view; 202 accepted.
GET /analytics/summary?from=...&to;=...	User	Own views, clicks, top links and daily trend.
GET /health/live	Infra/public	Process liveness check.
GET /health/ready	Infra	Database and app readiness check.
GET /docs	Protected or non-production	OpenAPI / Swagger documentation.

5. Detailed application flows

5.1 Registration and onboarding

1. Browser submits name, email and password to POST /auth/register.
2. Route applies rate limiting and validation middleware.
3. Auth controller calls authService.register().
4. Service normalizes email, checks duplicate account, hashes password and creates User.
5. Service creates a draft Profile for that user. A unique index protects username uniqueness.
6. API returns safe user data and short-lived access token; refresh token is set in a Secure, HttpOnly cookie.
7. Frontend sends the user to onboarding to choose username, add profile details and create the first link.
8. User previews the public page and publishes it.

5.2 Login, refresh and logout

```
LOGIN
```

```
Client -> POST /auth/login
       -> validate email/password
       -> compare password hash
       -> issue short-lived access JWT
       -> set refresh token as HttpOnly cookie

AUTHENTICATED REQUEST
Client -> Authorization: Bearer <access-token>
API    -> authenticate middleware verifies signature + expiry
       -> controller -> service -> model

ACCESS TOKEN EXPIRES
Client -> POST /auth/refresh (cookie sent by browser)
API    -> verify stored refresh-token hash
       -> rotate refresh token
       -> issue new access JWT + new cookie

LOGOUT
Client -> POST /auth/logout
API    -> revoke refresh session + clear cookie
```

Security note: Keep access tokens short-lived. Store refresh tokens only as hashes server-side and rotate them. For multi-device sessions, replace the single refreshTokenHash field with a Sessions collection. Cookie-based refresh endpoints need a deliberate CSRF strategy.

5.3 Create a link

1. Authenticated user submits title and URL to POST /links.

2. Validation middleware checks field types, length and URL format.

3. linkController.createLink() calls linkService.createLink().

4. Service finds the user's profile and assigns the next position.

5. Service creates the link with both userId and profileId ownership references.

6. Controller returns the created link. Dashboard refreshes the link list and preview.

5.4 Public profile visit and click

1. Visitor opens /:username in the frontend.

2. Frontend calls GET /public/profiles/:username.

3. publicService finds a published profile and returns safe public fields plus active links sorted by position.

4. Frontend renders the selected theme and links.

5. Frontend sends a best-effort page-view event.

6. When a visitor clicks a link, frontend records a click event and navigates to the destination. Analytics failure must never block navigation.

5.5 Reorder links

1. Frontend sends the complete ordered ID list to PATCH /links/reorder.

2. Service loads the current user's links for the current profile.

3. Service verifies every submitted ID belongs to that user/profile, with no duplicates and no missing IDs.

4. Service updates position values in one controlled operation.

5. API returns the newly ordered list.

5.6 Publish and unpublish

Publishing checks that the user has a valid username and display name. Decide whether V1 requires at least one active link. On unpublish, the public profile endpoint must stop serving the page immediately. Owner preview remains available behind authentication.

6. Model, controller, route and service responsibility map

Feature	Model	Controller	Route	Service
Auth	User	auth.controller	auth.routes	auth.service
Profile	Profile	profile.controller	profile.routes	profile.service
Links	Link	link.controller	link.routes	link.service
Public page	Profile + Link	public.controller	public.routes	public.service
Analytics	AnalyticsEvent	analytics.controller	analytics.routes	analytics.service

Example request path: **PATCH /api/v1/links/:linkId** → link.routes.ts → authenticate middleware → validate middleware → link.controller.ts → link.service.ts → Link model → MongoDB → controller response.

7. Frontend plan after backend completion

Frontend route	Purpose	Backend dependency
/register	Create account	POST /auth/register
/login	Sign in	POST /auth/login
/onboarding	Choose username and profile basics	GET/PATCH /profiles/me, username check
/dashboard	Overview and publish status	GET /auth/me, GET /profiles/me
/dashboard/profile	Edit profile	PATCH /profiles/me
/dashboard/links	Create/edit/toggle/reorder links	GET/POST/PATCH/DELETE /links
/dashboard/appearance	Choose theme	PATCH /profiles/me
/dashboard/analytics	Views and clicks	GET /analytics/summary
/:username	Public page	GET /public/profiles/:username

Frontend technology and structure

• React + TypeScript + Vite + Tailwind CSS.

• TanStack Query for API/server state; small client store for local UI state.

• One centralized API client; shared request/response types generated or aligned with OpenAPI.

• Route guards improve navigation experience, but backend authorization remains authoritative.

```
frontend/src/
  app/          router.tsx, providers.tsx
  features/
    auth/
    onboarding/
    dashboard/
    profile/
    links/
    appearance/
    analytics/
  components/
    ui/
    layout/
  lib/          api-client.ts, query-client.ts
  hooks/
  types/
  styles/
```

8. Validation and authorization rules

Input / action	Rule
Email	Trim and lowercase; validate format; unique database index.
Password	Set minimum length and strength policy; hash with Argon2id or bcrypt.
Username	Lowercase; URL-safe characters; length limits; reserved names blocked.
Link URL	Allow http:// and https:// only; reject javascript:, data: and other schemes.
Theme and icon	Allowlist known IDs; never accept arbitrary HTML or executable CSS.
Link update/delete	Query by both link ID and authenticated owner/profile ID.
Analytics read	Resolve profile from authenticated user; never accept arbitrary profileId as authority.
Public profile	Return only published profile and active links; never expose email or internal fields.

9. Security and production checklist

• Use Helmet, explicit CORS origin allowlist, HTTPS and secure cookie flags in production.

• Rate-limit register, login, refresh, password reset and public analytics endpoints.

• Use Zod validation for every request body, query and route parameter.

• Use centralized error middleware; do not expose stack traces or secrets to clients.

• Redact passwords, access tokens, refresh tokens and cookies from logs.

• Use least-privilege MongoDB credentials, backups and a tested restore process.

• Use request IDs and structured logs; add error monitoring and readiness checks.

• Define privacy notice, analytics retention and account deletion behavior before public launch.

• Use object storage for avatars; validate MIME type, file size and image dimensions.

10. Testing plan

Test type	Must verify
Unit	Username normalization, URL validation, publish rules, token helpers.
Integration	Register/login/refresh/logout; profile CRUD; link CRUD and reorder.
Authorization	User A cannot view or edit User B private resources.
Public endpoints	Unpublished profile returns not found; inactive links are hidden.
Abuse/security	Invalid URL schemes, brute-force limits, expired/reused refresh token.
End-to-end	Register → onboarding → add links → publish → open public page → click link.
Resilience	Database outage and analytics failure are handled without leaking secrets.

11. Development order and task checklist

Order	Task ID	Implementation task	Exit condition
1	BE-01	Initialize Express + TypeScript project, scripts and linting.	App compiles and starts.
2	BE-02	Environment config, Mongo connection, live/ready health routes.	DB connection and health checks work.
3	BE-03	Request ID, logger, async handler, error and 404 middleware.	Consistent errors and request logs.
4	BE-04	User/Profile models, schemas and indexes.	Indexes and model validation tested.
5	BE-05	Register, login, refresh, logout and /auth/me.	Auth integration tests pass.

Order	Task ID	Implementation task	Exit condition
6	BE-06	Authentication middleware and ownership helpers.	Protected route and cross-user tests pass.
7	BE-07	Profile CRUD, username check, publish/unpublish.	Profile flow works.
8	BE-08	Link model and create/list/update/delete.	Link CRUD tests pass.
9	BE-09	Reorder endpoint and ownership validation.	Order is stable and safe.
10	BE-10	Public profile endpoint and click/view event capture.	Public page data is correct.
11	BE-11	Analytics summary endpoint.	Owner-only analytics works.
12	BE-12	OpenAPI docs and API collection.	All V1 endpoints documented.
13	BE-13	Rate limits, security headers, CORS and integration tests.	Security checklist passes.
14	FE-01+	Build frontend routes and features against stable API.	Creator journey passes E2E.
15	REL-01	CI/CD, staging, monitoring, production and backups.	Release smoke tests pass.

12. Environment variables

```
NODE_ENV=development
PORT=5000
MONGODB_URI=
ACCESS_TOKEN_SECRET=
ACCESS_TOKEN_TTL=15m
REFRESH_TOKEN_TTL_DAYS=7
FRONTEND_ORIGIN=http://localhost:5173
COOKIE_DOMAIN=
COOKIE_SECURE=false
LOG_LEVEL=debug
SENTRY_DSN=
```

Keep only placeholders in .env.example. Never commit real secrets. Production values belong in the hosting platform's secret/environment configuration.

13. API payload examples

Register

```
POST /api/v1/auth/register
{
  "name": "Gobinda",
  "email": "gobinda@example.com",
  "password": "A-strong-password"
}

201 Created
{
  "success": true,
  "data": {
    "user": {
      "id": "USER_ID",
      "name": "Gobinda",
      "email": "gobinda@example.com"
    },
    "accessToken": "SHORT_LIVED_ACCESS_TOKEN"
  }
}
// Refresh token is set in an HttpOnly cookie.
```

Create a link

```
POST /api/v1/links
Authorization: Bearer <access-token>

{
  "title": "GitHub",
  "url": "https://github.com/example",
  "icon": "github"
}
```

```
201 Created
{
  "success": true,
  "data": {
    "link": {
      "id": "LINK_ID",
      "title": "GitHub",
      "url": "https://github.com/example",
      "position": 0,
      "isActive": true
    }
  }
}
```

Public profile

```
GET /api/v1/public/profiles/gobinda

200 OK
{
  "success": true,
  "data": {
    "profile": {
      "username": "gobinda",
      "displayName": "Gobinda",
      "bio": "Developer building useful things",
      "theme": "midnight",
      "links": [
        {
          "id": "LINK_ID",
          "title": "GitHub",
          "url": "https://github.com/example",
          "icon": "github"
        }
      ]
    }
  }
}
```

14. V1 decisions to lock before implementation

Decision	V1 recommendation
Initial audience	Creators and developers; validate this audience before broad expansion.
Username policy	Unique lowercase username; define reserved names and old-username policy.
Profile publishing	Require valid username and display name; decide if at least one active link is mandatory.
Refresh sessions	Single session for simplest V1, or Sessions collection if multi-device login is required.
Analytics privacy	Aggregate views/clicks; avoid storing raw IP addresses.
Themes	A few curated themes with safe tokens, not custom HTML/CSS.
Images	Object storage; persist URL/key, not binary image data in profile document.

15. First implementation session

1. Create the Git repository and backend directory.
2. Initialize Node.js, TypeScript and Express.
3. Create app.ts and server.ts with a clean startup/shutdown path.
4. Add env.config.ts and database.config.ts.
5. Add GET /health/live and GET /health/ready.
6. Add request ID, logger, not-found and centralized error middleware.
7. Commit the foundation before implementing authentication.

Recommended first milestone: a running Express + TypeScript API connected to MongoDB, with health checks, structured logs and consistent error responses. Only after this foundation is stable should we build User and Profile models, then authentication.

Appendix: Production release gate

• All P0 backend tests pass and API documentation matches implementation.

• No cross-user access paths; ownership checks tested.

• HTTPS, secure cookies, CORS allowlist and rate limits verified.

• Database indexes, backup and restore process tested.

• Public profile is responsive and does not leak private fields.

• Analytics failure does not block destination navigation.

• Privacy policy, terms and support contact are available.

• Monitoring, alerting and rollback plan are ready.